Local-First Khata: Evaluating Change-Tracking and Invariant-Preserving Synchronisation for Offline Credit Ledgers under Intermittent Connectivity and Clock Skew

Research Project Proposal

Abstract

Kirana stores in India still run a large share of their customer credit (khata) on paper, and the apps that aim to replace it must keep working when mobile connectivity drops. Offline-first apps meet this need by writing to a local database and synchronising later. Their correctness then rests on a small sync protocol that few people examine. An inspection of SmartOps, an offline-first point-of-sale and khata app built by the researchers, found that its pull/push protocol uses the client's write time as the change-tracking cursor. Under ordinary offline use this can stop records from ever reaching other devices, and it can leave a customer's balance different on two phones. This study asks which synchronisation design keeps a multi-device offline ledger complete, convergent and invariant-safe. It combines three methods. The first is formal modelling of the protocol in TLA+ with bounded model checking. The second is a factorial simulation experiment in which a headless multi-device simulator drives the real SmartOps backend under controlled offline periods, clock skew and device counts. The third is an invariant-confluence analysis of kirana operations. Four cursor strategies and two push protocols are compared on replication completeness, time to convergence, staleness, ledger divergence, invariant violations and payload overhead. The expected contribution is a verified protocol, LFK-Sync, together with evidence on how much each design choice matters.

Keywords: offline-first systems, local-first software, data synchronisation, CRDT, hybrid logical clocks, invariant confluence, TLA+, credit ledger, kirana retail

1. Introduction

1.1 Title of the Research

Local-First Khata: Evaluating Change-Tracking and Invariant-Preserving Synchronisation for Offline Credit Ledgers under Intermittent Connectivity and Clock Skew

The title names the object of study (synchronisation of offline credit ledgers), the independent variables (change-tracking strategy and invariant-preserving mechanism), and the conditions under which they are tested (intermittent connectivity and clock skew). "Local-first khata" is the name used in this proposal for the target design.

1.2 Background of the Study

India's small retail sector is dominated by kirana stores: family-run neighbourhood shops that sell groceries and daily goods on thin margins. A defining feature of the kirana relationship is informal credit. Regular customers take goods now and settle at the end of the week or month, and the shopkeeper records the debt in a handwritten khata book. Digitising this record has clear value. Adhikary et al. found that adopting digital payment technology raised revenue by 9.6% on average across 403 Indian unorganised retailers [1], and Choudhary and Aithal argue that low-cost technologies can change how kiranas compete [2]. A digital khata is only useful, however, if the shopkeeper can trust it more than the paper book it replaces.

Trust is hard to earn on Indian mobile networks. Coverage has grown quickly, but the quality and continuity of connections remain uneven, and the gap between coverage and actual use is still large in lower- and middle-income countries [3]. A shop cannot stop billing because the signal has dropped. Offline-first software answers this by writing every transaction to a database on the phone and exchanging changes with a server whenever a connection is available. The approach goes back to disconnected operation in the Coda file system [4] and the Bayou replicated store [5], and has recently been restated as "local-first software" [6].

The price of offline-first design is that there is no longer one copy of the truth. Each phone holds a replica, the server holds another, and a sync protocol has to reconcile them. When a shop runs two phones (the owner's and a helper's, say), both can record sales and repayments while offline, and both must end up agreeing on what every customer owes. Distributed-systems research has clear theory on when such agreement is possible. Physical clocks cannot be trusted to order events [7], conflict-free replicated data types (CRDTs) converge without

coordination [8], and invariant confluence says which business rules can survive concurrent offline updates [9]. In practice, though, most mobile apps use a sync framework's default protocol, and the correctness of that protocol is rarely tested.

SmartOps, the testbed for this study, shows the problem concretely. It is a React Native app using WatermelonDB, with a Node.js/Express and PostgreSQL backend deployed on AWS EC2. On inspection, its pull endpoint returns rows whose `updated_at` is greater than the device's last pull time, but `updated_at` holds the time the row was written *on the client*. A sale recorded offline at 11:00 and uploaded at 15:00 keeps its 11:00 timestamp. Any other device that pulled at, say, 12:00 will ask only for rows newer than 12:00 and will never receive that sale. The framework's own documentation warns informally about related race conditions and recommends a server-side timestamp [10], but no study has measured how often such anomalies occur or what they cost. The same inspection found an all-or-nothing push that one bad record can block indefinitely, and floating-point amounts on the phone set against fixed-precision amounts on the server.

1.3 Significance of the Study

Academic value. The study connects three bodies of theory that are usually studied apart: clock and cursor semantics [7], [11], CRDTs [8], [12], and invariant-preserving weak consistency [9], [13], [14]. It applies them to one concrete, commercially relevant workload. It also offers a replicable method that combines model checking with a factorial simulation experiment. Prior mobile-sync work, such as Simba [15], evaluated system throughput and consistency options but did not quantify cursor-induced anomalies.

Technical value. The study will produce a TLA+ specification of a widely used pull/push sync pattern, an open simulator that speaks the real protocol, and a corrected protocol (LFK-Sync). Each design choice will be backed by measured costs. Developers of other offline-first apps built on similar frameworks can reuse all three.

Practical value. For a shopkeeper, a missing repayment is not a technical inconvenience; it becomes a dispute with a regular customer. A protocol with a checked guarantee of completeness and balance agreement across devices removes one reason to keep the paper khata "just in case".

Beneficiaries. Kirana owners and their staff; developers of offline-first apps for MSMEs, pharmacies, micro-lenders and field workers; and researchers in distributed systems and ICT for development.

Contribution to knowledge. The study contributes (i) a measured account of how change-tracking strategy, offline duration and clock skew affect replication completeness in pull/push sync, (ii) an invariant-confluence classification of kirana operations, and (iii) evidence on the overhead of making the protocol safe.

SDG alignment. The work supports **SDG 8.3** (growth and formalisation of micro and small enterprises) and **SDG 8.10** (access to financial services), since a reliable digital khata is a first step towards formal records of informal credit. It also supports **SDG 9.c** (access to information and communication technology), by making software dependable on the networks people actually have.

1.4 Aim, Objectives, Research Questions and Hypotheses

Aim. To determine, through formal modelling and controlled simulation, which synchronisation design keeps a multi-device offline khata ledger complete, convergent and invariant-safe under intermittent connectivity and clock skew, and to propose and verify such a design.

Objectives.

- **O1.** To specify the SmartOps pull/push protocol in TLA+ and use the TLC model checker to find every violation of the completeness and convergence properties in a bounded model (up to 3 devices and 4 writes per device).
- **O2.** To measure replication completeness, time to convergence, staleness and ledger-balance divergence for four change-tracking strategies across 3 offline-duration levels, 3 clock-skew levels and 3 device counts, with at least 30 seeded runs per condition.
- **O3.** To classify the invariants of the kirana workload for invariant confluence and measure the violation rate of the non-confluent ones, with and without a stock-reservation (escrow) mechanism.
- **O4.** To compare the current all-or-nothing push with a chunked, partial-acknowledgement push under fault injection. The measures are the probability of a permanent sync stall and the time needed to drain an offline backlog.
- **O5.** To design LFK-Sync, verify its safety properties in TLA+, and test whether its payload and latency overhead is within 10% of the current protocol.

Research questions.

- **RQ1** (O1). Which interleavings of offline writes, pushes and pulls cause the current protocol to lose updates or diverge, and what is the smallest counterexample?
- **RQ2** (O2). How does change-tracking strategy affect replication completeness and ledger divergence, and how do offline duration, clock skew and device count moderate this effect?
- **RQ3** (O3). Which kirana invariants can be preserved without coordination, and how often are the others violated under realistic offline use?
- **RQ4** (O4). How does push-protocol design affect sync liveness when faults occur?
- **RQ5** (O5). Can completeness, convergence and invariant safety be guaranteed at an overhead acceptable for low-end phones?

Hypotheses.

- **H1.** Replication completeness differs across change-tracking strategies. The client-write-time cursor will show a non-zero missed-update rate, while server-sequence and HLC cursors will reach full completeness. (*$H1_0$: completeness does not differ across strategies.*)
- **H2.** Under the client-write-time cursor, the missed-update rate increases with offline duration and with negative clock skew. (*$H2_0$: offline duration and skew have no effect on the missed-update rate.*)
- **H3.** The append-only ledger model yields zero balance divergence after convergence under every strategy that delivers all records. Stock non-negativity is violated at a rate that rises with offline duration and device count unless escrow is used. (*$H3_0$: violation rates do not differ with and without escrow.*)
- **H4.** The chunked partial-acknowledgement push has a lower stall probability than the all-or-nothing push under fault injection. (*$H4_0$: stall probability is equal.*)
- **H5.** LFK-Sync's median payload and sync latency are non-inferior to the current protocol within a 10% margin. (*$H5_0$: overhead exceeds 10%.*)

2. Literature and Research Foundation

2.1 Literature Review

Disconnected and weakly connected operation. Kistler and Satyanarayanan showed with Coda that clients can keep working through server outages and reintegrate their changes later,

and that conflicts, though rare, must be detected and handled [4]. Bayou went further: each replica keeps an ordered write log, and applications supply their own conflict detection and merge procedures, so replicas move towards eventual consistency even over poor links [5]. Both systems were built for laptops on 1990s networks. They showed the approach was feasible, but they did not measure how often changes go missing when the change-tracking mechanism is wrong.

Consistency, clocks and staleness. Lamport's happened-before relation made the basic point that physical time cannot reliably order events across machines [7]. Hybrid logical clocks (HLC) combine a logical counter with physical time. This captures causality while staying close to wall-clock time, and it tolerates bounded skew [11]. Bailis et al. quantified staleness in eventually consistent quorum stores and found it was usually short [16]. Their work measures staleness *given* correct delivery; it does not address a cursor that never delivers some rows at all.

Conflict-free replication. Shapiro et al. defined CRDTs and Strong Eventual Consistency: if concurrent updates commute, or merge through a semilattice join, replicas converge with no coordination [8]. Later work made CRDTs practical. JSON CRDTs merge nested documents on the client [17], delta-state CRDTs send small incremental states [12], and join decompositions remove redundant deltas [18]. Kleppmann et al. argue that CRDTs are the key enabler for local-first software, which works offline, syncs across devices and leaves users in control of their data [6]. That essay is largely a design argument; it does not evaluate business invariants such as "stock may not go negative".

Invariants under weak consistency. Bailis et al. introduced invariant confluence as a necessary and sufficient condition for preserving an invariant without coordination, and showed that many real integrity constraints qualify [9]. Balegas et al. built Indigo, which adds reservations (escrow) to preserve invariants that do not qualify, at close to eventual-consistency latency [13]. Gotsman et al. gave a proof rule for deciding whether a mix of strong and weak operations is enough to preserve an invariant [14]. All three evaluate server-side, datacentre-scale workloads such as TPC-C; none examines phones that stay offline for hours.

Mobile data sync. Go et al. studied how popular mobile apps handle sync and found data loss and inconsistency under network and device failures. They proposed Simba, a sync service with unified tabular and object data and selectable consistency [15]. This is the

closest prior work. It predates current JavaScript offline-first frameworks, however, and did not isolate cursor semantics or financial ledgers.

Formal methods in industry. Newcombe et al. describe how Amazon Web Services used TLA+ and model checking to find subtle, serious bugs in production distributed designs that testing and review had missed [19]. The TLA+ language and the TLC checker are documented by Lamport [20]. Their use on mobile sync protocols is rare in the literature.

Kirana digitisation. Management research documents the benefit of digital tools for unorganised retailers [1], [2] and the persistence of connectivity gaps [3]. It treats the software as a black box and does not ask whether offline data stays correct.

Table 1 summarises the studies most directly relevant to this proposal.

Table 1. Summary of key prior studies

Study	Focus	Method	Key finding	Limitation for this problem
Kistler & Satyanarayanan [4]	Disconnected file access	System build, usage study	Offline work and reintegration feasible	Files, not ledgers; no cursor analysis
Terry et al. [5]	Weakly connected replication	System design	App-specific merge; ordered logs converge	No quantitative anomaly measurement
Shapiro et al. [8]	CRDT theory	Formal	Commutative updates converge without coordination	No application invariants
Bailis et al. [9]	Invariant confluence	Formal and experimental	Many invariants need no coordination	Datacentre workloads only
Balegas et al. [13]	Invariant-preserving EC	System and experiments	Escrow preserves invariants cheaply	Server replicas, not phones
Go et al. [15]	Mobile app sync	Study and system	Apps lose or corrupt data during sync	Pre-2015 native stack; cursor not isolated
Kleppmann et al. [6]	Local-first ideals	Essay and prototypes	CRDTs enable offline collaboration	No invariant or anomaly measurement
Newcombe et al. [19]	Formal methods at AWS	Industrial experience	TLA+ finds deep design bugs	Not applied to mobile sync

2.2 Theories, Concepts and Existing Methodologies

Happened-before and logical time [7]. Event *a* happens before *b* if they occur in sequence on one process, or if *a* sends a message that *b* receives. Wall-clock timestamps may contradict this order. The theory predicts that a cursor built on client clocks can skip events, which is exactly the anomaly under study.

Hybrid logical clocks [11]. An HLC timestamp is a pair: the largest physical time seen so far, and a logical counter. It is monotonic, respects causality and stays within the clock-synchronisation error of real time. It is a candidate cursor that needs no central sequence.

Change data capture and cursors [21]. A pull-based sync client asks for "everything after position *p*". This is correct only if positions are assigned in the order in which changes become *visible* on the server. A server-assigned, commit-ordered sequence satisfies this condition; a client timestamp does not.

Strong Eventual Consistency and CRDTs [8]. Replicas that have received the same set of updates are in the same state. An append-only ledger is a grow-only set, which is a CRDT, and the balance derived from it (Σ credit − Σ repayment) is therefore convergent by construction. SmartOps already models khata and stock this way. The theory predicts zero balance divergence once delivery is complete.

Invariant confluence [9]. An invariant *I* is confluent under a set of operations if any two *I*-valid states that diverge from a common ancestor can be merged into an *I*-valid state. "Balance equals the sum of entries" is confluent. "Stock ≥ 0" under concurrent decrements is not. Non-confluent invariants need coordination, or an escrow scheme that pre-allocates rights to decrement [13].

Existing methodologies. Three methods are used in this literature. *Model checking* [19], [20] explores every interleaving of a small model exhaustively. It finds deep bugs but cannot measure frequency. *Simulation and benchmark experiments* [16], [18] measure frequency and cost under chosen workloads, but they cover only the scenarios that are generated. *Field or trace studies* [15] give realism but little control. Combining the first two, with parameters drawn from real traces, uses the strength of each to cover the other's weakness.

Table 2. Variables drawn from the literature

Variable	Type	Source	Operationalisation in this study
Change-tracking strategy	Independent	[7], [11], [21]	4 levels: client write-time (baseline), server receive-time, server commit-sequence, HLC
Invariant mechanism	Independent	[9], [13]	None vs escrow for stock
Push protocol	Independent	[15]	All-or-nothing vs chunked partial-ack
Offline duration, clock skew, device count	Moderators	[4], [5]	3 levels each (Section 3.3)

Variable	Type	Source	Operationalisation in this study
Replication completeness	Dependent	[8]	Share of server rows present on every replica at quiescence
Staleness, convergence time	Dependent	[16]	ms from commit to visibility on all replicas
Ledger divergence	Dependent	this study	Σ absolute per-customer balance difference (₹) vs server
Invariant violations	Dependent	[9]	Count of negative-stock states per 1,000 sales
Stall probability, drain time	Dependent	[15]	Share of runs with a backlog that never drains; time to drain
Payload, latency overhead	Dependent	[18]	Bytes and ms per synced record

2.3 Research Gap

The literature leaves four gaps that this study addresses.

1. **No measurement of cursor anomalies in mobile pull/push sync.** Clock theory [7], [11] predicts that client-time cursors are unsafe, and framework documentation hints at it [10]. No study has measured how often records are lost, or how the loss rate grows with offline duration and skew.
2. **Invariant theory has not been applied to mobile ledgers.** Invariant confluence and escrow [9], [13], [14] were evaluated on datacentre replicas with sub-second partitions, not on phones that stay offline for hours and hold financial records.
3. **The consequences are not stated in financial terms.** Mobile-sync studies report errors as generic inconsistency [15], not as money owed that differs between two devices, which is the unit a shopkeeper cares about.
4. **Formal verification of offline-first app protocols is missing.** TLA+ has proven its value in industry [19], but no published specification of the timestamp-cursor pull/push pattern used by current JavaScript offline-first frameworks could be found.

Proposed gap solution. The study first uses model checking to establish that the anomalies exist and to find minimal counterexamples. It then uses a factorial simulation against the real backend to measure how frequent and how costly they are. Finally it proposes LFK-Sync, which is verified with the same model and tested with the same simulator, so that the fix is evaluated under the conditions that exposed the problem.

2.4 Research Framework / Conceptual Framework

Figure 1 shows the conceptual framework. The *synchronisation design* has four components: change-tracking strategy, conflict/data model, invariant mechanism and push protocol. These are the independent variables. They act on the *dependent variables*, which fall into three groups: correctness (completeness, ledger divergence, invariant violations), timeliness (staleness, convergence time) and cost (payload, latency, stall probability). *Operating conditions* moderate these effects: offline duration, clock skew, device count, write rate and fault rate. Hardware, dataset and random seeds are held constant as control variables.

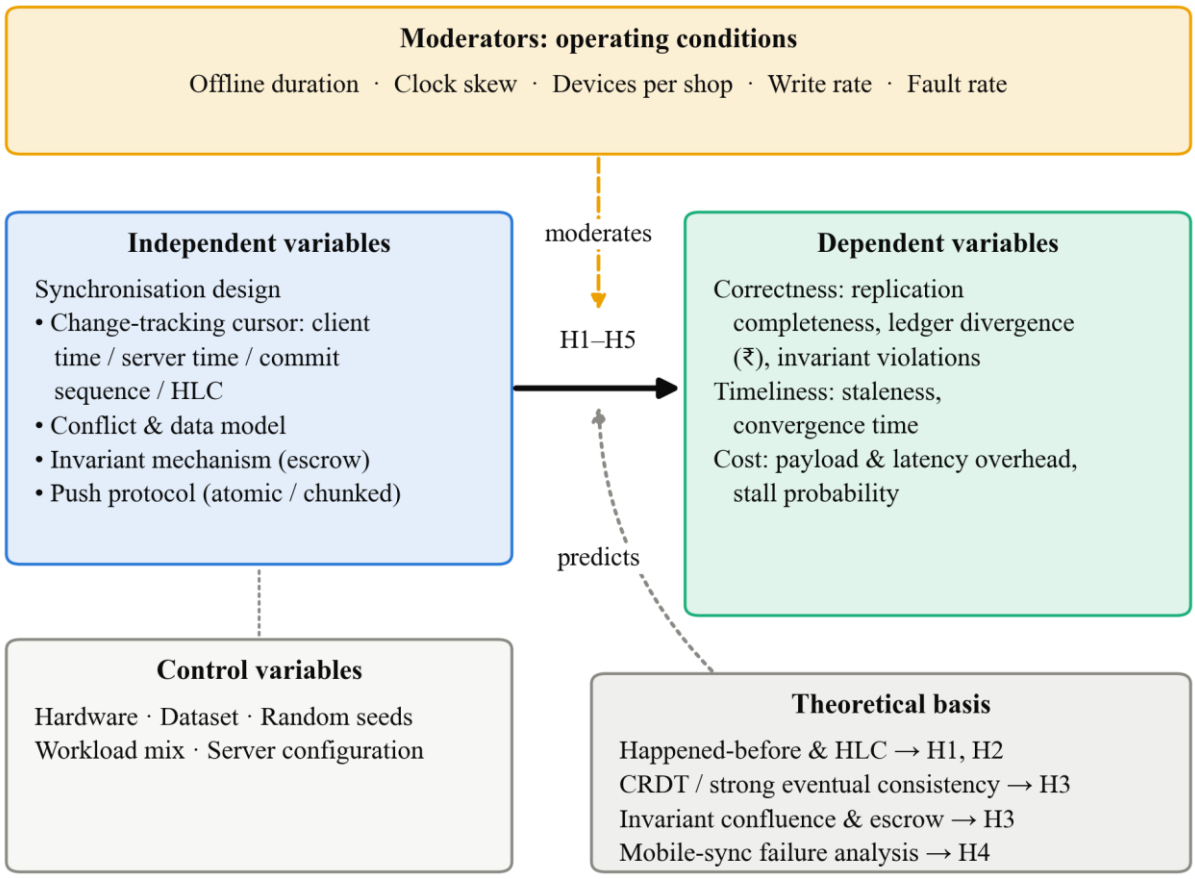


Fig. 1. Conceptual framework: synchronisation design choices (IVs) affect correctness, timeliness and cost outcomes (DVs), moderated by operating conditions and grounded in clock, CRDT and invariant-confluence theory

The framework rests on three theoretical links. Clock theory [7], [11] predicts the effect of the cursor strategy on completeness (H1) and its interaction with skew (H2). CRDT theory [8] predicts that the append-only ledger converges once delivery is complete (H3, first part). Invariant confluence [9] predicts that stock non-negativity will fail without coordination and that escrow can prevent this (H3, second part). The link from push protocol to stall probability (H4) follows from the failure analysis of Go et al. [15].

3. Research Problem and Design

3.1 Research Problem

Offline-first khata apps must keep every replica of a customer's credit record complete and consistent. In practice, their correctness depends on a sync protocol that is neither specified nor tested. In SmartOps, the protocol takes its cursor from client clocks, commits an entire push as a single all-or-nothing transaction, and applies last-arrival-wins to mutable records. It does not coordinate stock decrements across devices. The existing deficiency is that no one knows how often these choices cause lost records, disagreeing balances, negative stock or stuck devices under real Indian connectivity, nor what a verified alternative would cost. The impact is financial and social: a missing repayment or a double-counted credit sale becomes an argument with a customer, and a shopkeeper who has seen one such error will return to paper.

Problem statement. *It is not known how change-tracking strategy, invariant mechanism and push protocol affect the completeness, convergence and invariant safety of a multi-device offline credit ledger under intermittent connectivity and clock skew, nor whether a formally verified protocol can remove these failures at an acceptable cost on low-end devices.*

3.2 Research Approach

The study takes a **quantitative** approach. Every research question concerns measurable quantities: counts of missing rows, rupee differences, milliseconds, bytes and probabilities. The hypotheses are directional and can be tested statistically. A qualitative component would add little, because the phenomenon is a property of the protocol, not of user perception. The approach sits within the positivist tradition of experimental computer systems research, where claims are backed by controlled, repeatable measurement [16], [18], supplemented by formal proof where exhaustive reasoning is possible [19].

3.3 Research Design

The design is a **mixed formal–experimental design**, with a development component, in three stages.

- **Stage A: Formal analysis (O1, O5).** The researchers will write TLA+ specifications of the baseline protocol and of LFK-Sync. Safety properties (no lost update, balance agreement at quiescence, stock ≥ 0 under escrow) and a liveness property (every pushed record is eventually pulled by every connected device) will be checked with TLC in

bounded models. This is an *analytical* design, chosen because it is the only method that covers every interleaving and yields minimal counterexamples.

- **Stage B: Controlled simulation experiment (O2–O4).** A **full factorial experiment** will run a headless multi-device simulator against a local instance of the real SmartOps backend and PostgreSQL. The simulator reuses the app's sync payload format and endpoints. It schedules connectivity with a two-state (online/offline) Markov model whose parameters are fitted to traces collected for this study (Section 4.1). A *simulation* design is chosen because it controls the moderators precisely, repeats cheaply with different seeds, and uses the production server code rather than a model of it.
- **Stage C: Development and validation (O5).** LFK-Sync will be implemented in the SmartOps codebase. Its behaviour will be checked against the simulator results in a short validation on one low-end Android phone and one emulator, running scripted offline scenarios.

Table 3. Factorial design for Stage B

Factor	Levels	Values
Change-tracking strategy	4	Client write-time (baseline); server receive-time; server commit-sequence; HLC
Mean offline period	3	5 min; 1 h; 8 h
Device clock skew	3	0 s; ±2 min; ±30 min
Devices per shop	3	2; 3; 5
Replications	30 per cell	Different random seeds
Total runs	**3,240**	Each ~1 simulated trading day at 5–15 sales/h per device

O3 and O4 run as smaller follow-up designs on the best and baseline strategies: escrow on/off × offline period × devices, and push protocol × fault type × fault rate.

3.4 Scope of the Research

- **Research area.** Replicated data management for offline-first mobile applications, applied to retail credit ledgers.
- **Population and unit of analysis.** The population is pull/push offline-first sync protocols of the kind used by JavaScript mobile frameworks. The unit of analysis is a simulated shop running 2–5 devices over one trading day. SmartOps is the reference implementation.
- **Variables.** As listed in Table 2 and the factorial design above.

- **Conditions.** A single server region; mobile devices on intermittent connections; offline periods from minutes to a working day; clock skew up to ±30 minutes, reflecting phones with manual or drifting time settings.
- **Applications.** Khata and inventory apps for kiranas and pharmacies. The results should also transfer to other offline-first record-keeping apps, such as field data collection or microfinance agent apps.
- **Exclusions.** Credit scoring and lending decisions; payment processing; peer-to-peer (device-to-device) sync; Byzantine or malicious clients; multi-shop and franchise hierarchies; user-interface design; and energy measurement.

4. Research Implementation and Evaluation

4.1 Data Collection

Primary data.

1. **Connectivity and clock traces.** A small logging build of SmartOps will run on 5–8 phones belonging to the researchers and consenting volunteers, for 14 days. It will record only network-state transitions (`NetInfo` events: connected/disconnected, network type) and the device's clock offset against the server's `/api/health` timestamp, sampled every 15 minutes. It will record no location, contacts or content. This gives roughly 1,000–2,000 connectivity transitions and 7,000–10,000 clock-offset samples. These are used to fit the Markov connectivity model and to set realistic skew levels.
2. **Experimental measurements.** Every simulator run logs, per sync cycle, the pull and push durations, bytes sent and received, records per table, HTTP status and the device's cursor. At quiescence it logs a full replica-versus-server comparison. Across 3,240 runs of the main design plus follow-ups, this is expected to yield about 4–6 million sync-cycle records.
3. **Model-checking outputs.** TLC state counts, violated properties and counterexample traces for each specification and model size.

Secondary data.

- The SmartOps demo and showcase seed datasets (products, customers, sales and khata entries for realistic shops). These define the product catalogue size, basket sizes and the credit-sale ratio used by the workload generator.

- Published connectivity statistics [3], used to sanity-check the fitted parameters.

Sample and data size justification. The planned 30 replications per cell will be confirmed by an a-priori power analysis on pilot runs, targeting power of 0.8 at $\alpha = 0.05$ for a small-to-medium effect.

4.2 Tools and Techniques

Table 4. Tools and techniques

Category	Item	Purpose
Specification	TLA+ with the TLC model checker (TLA+ Toolbox / VS Code extension) [20]	Formal models of baseline and LFK-Sync; safety and liveness checks
Testbed server	SmartOps backend (Node.js 20, Express 4), PostgreSQL 16, Docker Compose	Real sync endpoints, run locally for isolation; EC2 copy for the validation stage
Simulator	Custom Node.js discrete-event simulator (to be built)	Virtual devices with local replicas, a virtual clock with injected skew, a Markov connectivity model, and a workload generator
Fault injection	Toxiproxy (latency, timeouts, connection resets); crafted poison records	O4 liveness experiments
Algorithms	Server commit-sequence cursor (`bigserial` / transaction-ordered log with snapshot guard); HLC [11]; grow-only ledger CRDT [8]; escrow allocation for stock [13]; chunked push with per-chunk acknowledgement and a quarantine table	Strategies and LFK-Sync
Device validation	Android emulator (API 34) and one low-end Android phone (2–3 GB RAM), Expo dev build	Confirm simulator behaviour on the real client
Analysis	Python 3.12: pandas, SciPy, statsmodels, lifelines, matplotlib	Statistics and figures
Hardware	Researchers' laptops (≥16 GB RAM); existing AWS EC2 t3.micro	Simulation host; validation server

Figure 2 shows how the testbed fits together.

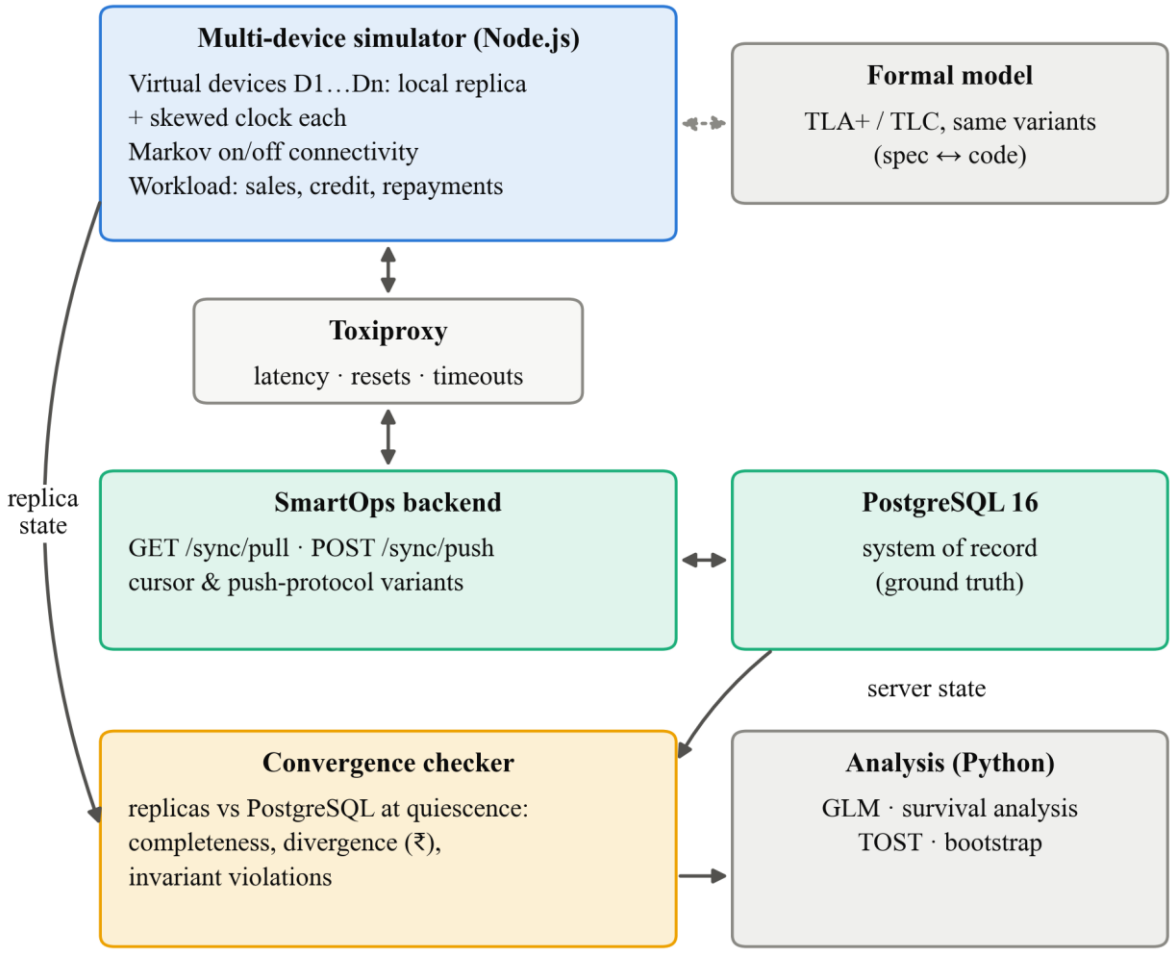


Fig. 2. Experimental testbed: the simulator drives virtual devices against the real SmartOps backend through a fault-injection proxy; a convergence checker compares every replica with PostgreSQL at quiescence

4.3 Data Analysis and Performance Measures

Performance metrics.

- *Replication completeness* C = (rows present on every replica) / (rows on the server), computed at quiescence.
- *Missed-update count*: the number of server rows absent from at least one replica after two full sync rounds while connected.
- *Time to convergence*: time from the last write to the point at which every replica matches the server. Runs that never converge are treated as censored.
- *Staleness*: the time from a row's server commit until it is visible on each other device, reported as median and p95.
- *Ledger divergence* D = Σ over customers of |balance on device − balance on server|, in ₹.

- *Invariant violation rate*: negative-stock states per 1,000 sales.
- *Stall probability*: the share of runs in which a device's backlog never drains; *drain time*: the time to empty a backlog after reconnection.
- *Overhead*: bytes and milliseconds per synced record, relative to the baseline.

Statistical techniques.

- **H1, H2.** Completeness is a proportion per run, so a **binomial generalised linear model** (logit link) will be fitted with strategy, offline period, skew, device count and their two-way interactions as predictors. Results will be reported as odds ratios with 95% CIs. A non-parametric **Kruskal–Wallis** test with **Dunn post-hoc** comparisons (Holm correction) will serve as a robustness check.
- **Convergence time. Kaplan–Meier** curves by strategy, compared with the **log-rank test**, and a **Cox proportional-hazards** model for the moderators. Survival analysis handles runs that never converge without discarding them.
- **H3.** Violation counts will be modelled with **Poisson or negative-binomial regression** (escrow × offline period × devices). Balance divergence under complete delivery is expected to be exactly zero, so it will be checked as an assertion across all runs rather than tested.
- **H4.** Stall probability will be compared with **Fisher's exact test** and reported with Wilson 95% CIs; drain time with the **Mann–Whitney U** test.
- **H5. Two one-sided tests (TOST)** for non-inferiority, with a 10% margin on median bytes and median latency per record, using bootstrap CIs (10,000 resamples).
- **Effect sizes** will be reported for every comparison (odds ratio, Cliff's δ, or hazard ratio), not only p-values. The significance level is $\alpha = 0.05$ with Holm correction within each hypothesis family.

Comparisons. Each of the three alternative strategies is compared with the baseline, LFK-Sync is compared with both the baseline and the best single-change strategy, and the simulator results are compared with the device-validation runs to check external validity.

4.4 Expected Outcomes

The following are expectations derived from theory and from the code inspection. They are not results.

- **Formal analysis.** TLC is expected to find a short counterexample for the baseline protocol, likely involving two devices and a single offline write, in which a pushed row is never pulled by the peer. LFK-Sync is expected to pass all safety properties in the bounded models.
- **Completeness.** The client write-time cursor is expected to fall below full completeness whenever writes are pushed after a peer's pull, with losses growing with offline duration and negative skew. The server commit-sequence and HLC strategies are expected to reach full completeness. Server receive-time is expected to fall in between, because of the race between the pull snapshot and concurrent commits.
- **Ledger.** Once delivery is complete, the append-only khata model is expected to show no balance divergence. The rounding difference between client floating-point values and server `NUMERIC(10,2)` is expected to appear as small but non-zero divergence until amounts are stored in integer paise.
- **Invariants.** Negative-stock events are expected to appear under coordination-free operation and to become rare or absent with escrow, at the cost of occasionally refusing an offline sale when a device's allocation runs out.
- **Liveness.** The all-or-nothing push is expected to stall permanently once a poison record exists, while the chunked protocol quarantines that record and drains the rest.
- **Proposed solution.** The study will deliver LFK-Sync: a server commit-sequence cursor with an HLC ordering field, amounts in integer paise, an append-only ledger CRDT, escrow for stock and chunked partial-acknowledgement push, with its TLA+ specification and the simulator released as reusable artefacts. The design target is completeness of 1.0 in every tested condition at no more than 10% overhead.

5. Research Management and Output

5.1 Limitations

- **Technical.** Model checking covers only bounded models (small numbers of devices and writes). Bugs that need larger configurations could be missed, and the simulation stage partly offsets this. The simulator drives the real server but simulates the client replica, so client-side storage behaviour (for example, persistence delays in the LokiJS adapter) is outside the main experiment.

- **Data.** Connectivity traces come from a small convenience sample of phones in one city and may not represent rural or other regional networks. Workloads are synthetic, parameterised from seed data, not logged from real shops.
- **Resource.** A single low-end phone and an emulator limit how widely the device validation generalises across Android hardware.
- **Time.** The 16-week plan fixes the factorial design at three levels per moderator; finer levels and longer simulated periods are left for future work.
- **Financial.** No budget is available for commercial device farms or for field deployment in shops.
- **Methodological.** Simulation gives strong internal validity but weaker external validity. The one-day simulated trading period may miss effects that build up over weeks, such as cursor drift after long idle periods.

5.2 Research Work Plan and Gantt Chart

The work is planned over 16 weeks from the date the proposal is approved.

Table 5. Work plan, milestones and deliverables

#	Activity	Weeks	Deliverable
1	Detailed literature review and problem refinement	1–3	Review chapter; final variable definitions
2	TLA+ specification of baseline protocol and model checking	2–5	Baseline spec; counterexample traces
3	Connectivity and clock trace collection	3–6	Anonymised trace dataset; fitted Markov parameters
4	Simulator, instrumentation and convergence checker	4–8	Validated simulator
5	Implement strategies, escrow, chunked push and LFK-Sync	6–10	Strategy branches in SmartOps
6	Factorial experiments and follow-up designs	9–12	Raw results dataset
7	TLA+ verification of LFK-Sync	10–12	Verified spec
8	Statistical analysis	11–14	Analysis notebooks and figures
9	Device validation	12–13	Validation report
10	Report writing and review	13–16	Final research report

Milestones: M1 (week 3): literature review complete; M2 (week 5): baseline counterexample found; M3 (week 8): simulator validated; M4 (week 12): all experiments complete; M5 (week 16): final report submitted.

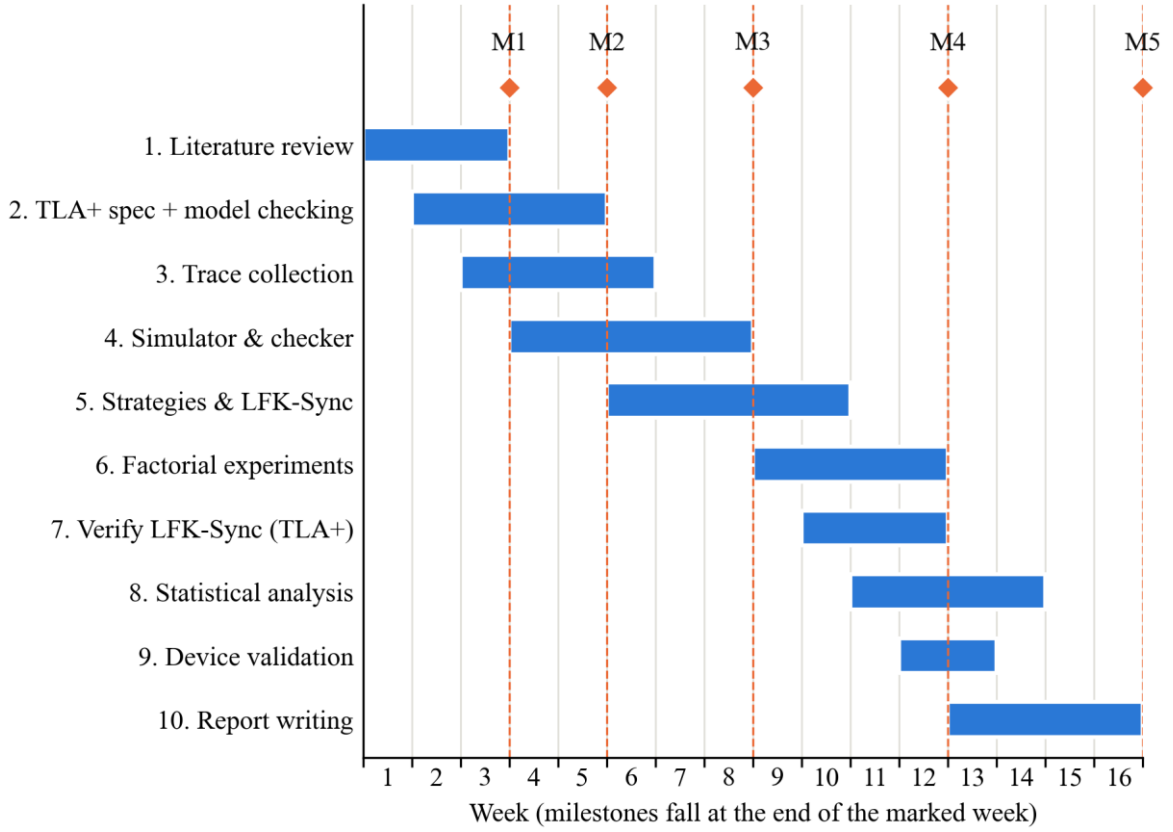


Fig. 3. Gantt chart of the 16-week work plan with milestones M1–M5

5.3 Cost Estimation / Budget

All software is open source. The existing EC2 instance and the researchers' laptops cover most computing needs. Figures are estimates in Indian rupees.

Table 6. Estimated budget

Head	Item	Qty × rate	Amount (₹)
Equipment	Low-end Android phone (2–3 GB RAM, refurbished) for device validation	1 × 6,500	6,500
Software	TLA+ tools, Node.js, PostgreSQL, Docker, Python libraries, Toxiproxy	Open source	0
Cloud	AWS EC2 t3.micro and storage for the validation stage	4 months × 900	3,600
Data collection	Mobile data packs for trace-logging volunteer phones	6 × 300	1,800

Head	Item	Qty × rate	Amount (₹)
Materials	Printing, binding, consent forms	Lump sum	1,200
Travel	Local travel for guide meetings and library access	Lump sum	1,000
Testing	Simulation runs on researchers' laptops	Existing hardware	0
Publication	Student registration at a national IEEE conference (optional)	1 × 7,500	7,500
Miscellaneous	Contingency (≈10% of the above)	—	2,200
Total			**23,800**

Without the optional conference registration, the total is ₹16,300.

5.4 Expected Contribution, Ethical Considerations and References

Expected contribution.

- *Knowledge.* The study will provide the first measured account of cursor-induced anomalies in mobile pull/push sync, and an invariant-confluence classification of kirana credit and stock operations.
- *Technical.* It will release a TLA+ specification of a common offline-first sync pattern, an open simulator that uses a real backend, and LFK-Sync as a verified protocol that other developers can adopt.
- *Practical.* A khata app whose balances are guaranteed to agree across the shop's phones, with sync that cannot be blocked by a single bad record.
- *Methodological.* A repeatable recipe for testing offline-first protocols: model checking for existence, factorial simulation for frequency and cost, and a small device study for validity.

Ethical considerations.

- *Approval.* The only human involvement is the connectivity-trace logging on volunteer phones. The protocol will be submitted to the institutional ethics committee before collection starts, and no data will be collected before approval.
- *Consent.* Volunteers will receive a one-page information sheet in English, Hindi or Marathi explaining exactly what is logged (network state and clock offset only) and will sign a consent form. They may withdraw at any time and have their data deleted.

- *Privacy.* No location, contacts, messages, app usage or ledger content is logged. Device identifiers are replaced with random study IDs on the phone before upload. Experimental ledger data is entirely synthetic, so no real customer's financial data is used.
- *Data security.* Traces are sent over HTTPS and stored in an access-controlled, encrypted folder that only the researchers can open. Raw traces will be deleted within six months of submission; only aggregated parameters will be kept. Handling follows the principles of India's Digital Personal Data Protection Act, 2023 [22]: purpose limitation, data minimisation and storage limitation.
- *Plagiarism.* All sources are cited in IEEE style. The final report will be checked with the institution's similarity-checking software, and every figure and table is original.

References

[1] A. Adhikary, K. S. Diatha, S. B. Borah, and A. Sharma, "How does the adoption of digital payment technologies influence unorganized retailers' performance? An investigation in an emerging market," *J. Acad. Marketing Sci.*, vol. 49, no. 5, pp. 882–902, 2021, doi: 10.1007/s11747-021-00778-y.

[2] V. Choudhary and R. K. Aithal, "Low-cost technologies for kirana (small) retail: A game changer?," *IIM Kozhikode Society & Management Review*, vol. 12, no. 2, pp. 250–258, 2023, doi: 10.1177/22779752221099128.

[3] GSMA, "The state of mobile internet connectivity report 2023," GSMA, London, U.K., Oct. 2023. [Online]. Available: https://www.gsma.com/r/wp-content/uploads/2023/10/The-State-of-Mobile-Internet-Connectivity-Report-2023.pdf

[4] J. J. Kistler and M. Satyanarayanan, "Disconnected operation in the Coda file system," *ACM Trans. Comput. Syst.*, vol. 10, no. 1, pp. 3–25, 1992.

[5] D. B. Terry, M. M. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer, and C. H. Hauser, "Managing update conflicts in Bayou, a weakly connected replicated storage system," in *Proc. 15th ACM Symp. Operating Systems Principles (SOSP)*, Copper Mountain, CO, USA, 1995, pp. 172–182, doi: 10.1145/224056.224070.

[6] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, Athens, Greece, 2019, pp. 154–178, doi: 10.1145/3359591.3359737.

[7] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978, doi: 10.1145/359545.359563.

[8] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Proc. 13th Int. Symp. Stabilization, Safety, and Security of Distributed Systems (SSS)*, LNCS, vol. 6976, 2011, pp. 386–400, doi: 10.1007/978-3-642-24550-3_29.

[9] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, "Coordination avoidance in database systems," *Proc. VLDB Endowment*, vol. 8, no. 3, pp. 185–196, 2014, doi: 10.14778/2735508.2735509.

[10] Nozbe, "Sync — Backend implementation," *WatermelonDB Documentation*. [Online]. Available: https://watermelondb.dev/docs/Sync/Backend (accessed Sep. 21, 2026).

[11] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone, "Logical physical clocks," in *Principles of Distributed Systems (OPODIS 2014)*, LNCS, vol. 8878, 2014, pp. 17–32, doi: 10.1007/978-3-319-14472-6_2.

[12] P. S. Almeida, A. Shoker, and C. Baquero, "Delta state replicated data types," *J. Parallel Distrib. Comput.*, vol. 111, pp. 162–173, 2018, doi: 10.1016/j.jpdc.2017.08.003.

[13] V. Balegas *et al.*, "Putting consistency back into eventual consistency," in *Proc. 10th European Conf. Computer Systems (EuroSys)*, Bordeaux, France, 2015, Art. no. 6, doi: 10.1145/2741948.2741972.

[14] A. Gotsman, H. Yang, C. Ferreira, M. Najafzadeh, and M. Shapiro, "'Cause I'm strong enough: Reasoning about consistency choices in distributed systems," in *Proc. 43rd*

ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages (POPL), St. Petersburg, FL, USA, 2016, pp. 371–384, doi: 10.1145/2837614.2837625.

[15] Y. Go, N. Agrawal, A. Aranya, and C. Ungureanu, "Reliable, consistent, and efficient data sync for mobile apps," in *Proc. 13th USENIX Conf. File and Storage Technologies (FAST)*, Santa Clara, CA, USA, 2015, pp. 359–372.

[16] P. Bailis, S. Venkataraman, M. J. Franklin, J. M. Hellerstein, and I. Stoica, "Probabilistically bounded staleness for practical partial quorums," *Proc. VLDB Endowment*, vol. 5, no. 8, pp. 776–787, 2012, doi: 10.14778/2212351.2212359.

[17] M. Kleppmann and A. R. Beresford, "A conflict-free replicated JSON datatype," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 10, pp. 2733–2746, 2017, doi: 10.1109/TPDS.2017.2697382.

[18] V. Enes, P. S. Almeida, C. Baquero, and J. Leitão, "Efficient synchronization of state-based CRDTs," in *Proc. IEEE 35th Int. Conf. Data Engineering (ICDE)*, Macao, China, 2019, pp. 148–159, doi: 10.1109/ICDE.2019.00022.

[19] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, "How Amazon Web Services uses formal methods," *Commun. ACM*, vol. 58, no. 4, pp. 66–73, 2015, doi: 10.1145/2699417.

[20] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Boston, MA, USA: Addison-Wesley, 2002.

[21] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[22] *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023), Gazette of India, Ministry of Law and Justice, Government of India, Aug. 11, 2023.